

Compte Rendu des TPs en PL/SQL

Module : PL/SQL

Prof : Pr. Himech

Étudiant : EL AZRAK Mohammed

TP - Sous Requêtes

0.0.1 Exercice 1

1.

```
1 SELECT Numetu, Nometu, Cdsexe
2 FROM ETUDIANT
3 WHERE Dtnaiss IS NULL;
```

2.

```
1 SELECT Nometu, Dtnaiss
2 FROM ETUDIANT
3 WHERE Nometu LIKE '_a%';
```

3.

```
1 SELECT Nomat
2 FROM MATIERE
3 WHERE Coeff BETWEEN 1 AND 2;
```

4.

```
1 SELECT Note
2 FROM NOTES
3 WHERE Numetu = 1
4 AND Note IN (SELECT Note FROM NOTES WHERE Numetu = 2);
```

5.

```
1 SELECT Note
2 FROM NOTES
3 WHERE Numetu = 1
4 AND Note > ANY (SELECT Note FROM NOTES WHERE Numetu = 2);
```

6.

```
1 SELECT Note
2 FROM NOTES
3 WHERE Numetu = 1
4 AND Note < ALL (SELECT Note FROM NOTES WHERE Numetu = 9);
```

7.

```
1 SELECT * FROM ETUDIANT
2 WHERE Numetu NOT IN (SELECT Numetu FROM NOTES);
```

8.

```
1 SELECT Cdsexe, AVG(MONTHS_BETWEEN(TO_DATE('01/01/2000', 'DD/MM/YYYY'),
2 Dtnaiss) / 12) AS Age_Moyen
3 FROM ETUDIANT
4 GROUP BY Cdsexe;
```

9.

```

1 SELECT Nomens, Grade
2 FROM ENSEIGNANT
3 WHERE Numens IN (SELECT Numens
4                   FROM MATIERE
5                   WHERE Nomat = 'Histoire');

```

10.

```

1 SELECT Numetu, Nometu
2 FROM ETUDIANT
3 WHERE Numetu NOT IN (SELECT Numetu
4                      FROM NOTES
5                      WHERE Numat IN (SELECT Numat
6                                     FROM MATIERE
7                                     WHERE Nomat = 'Sociologie'));

```

0.0.2 Exercice 2

11.

```

1 SELECT Lieu
2 FROM DEPT
3 WHERE NumDept IN (SELECT NumDept
4                   FROM EMP
5                   WHERE Commission IS NOT NULL AND Commission > 0);

```

12.

```

1 SELECT NomDept, Lieu
2 FROM DEPT
3 WHERE NumDept IN (SELECT NumDept
4                   FROM EMP
5                   WHERE Poste = 'Ingenieur');

```

13.

```

1 SELECT NomDept, Lieu
2 FROM DEPT
3 WHERE NumDept NOT IN (SELECT NumDept
4                       FROM EMP
5                       WHERE Poste = 'Ingenieur');

```

14.

```

1 SELECT E.NomE
2 FROM EMP E
3 WHERE E.Salaire < 0.5 * (SELECT S.Salaire
4                       FROM EMP S
5                       WHERE S.Matr = E.Sup);

```

15.

```

1 SELECT NomE
2 FROM EMP
3 WHERE Salaire > ALL (SELECT Salaire
4                       FROM EMP
5                       WHERE Poste = 'Commercial');

```

16.

```

1  -- Solution 1
2  SELECT NomE
3  FROM EMP E
4  WHERE Salaire > ALL (SELECT Salaire
5                        FROM EMP C
6                        WHERE C.Poste = 'Commercial'
7                        AND C.NumDept = E.NumDept);
8
9  -- Solution 2
10 SELECT NomE
11 FROM EMP E
12 WHERE NumDept IN (SELECT NumDept FROM EMP WHERE Poste = 'Commercial')
13    AND Salaire > ALL (SELECT Salaire
14                      FROM EMP C
15                      WHERE C.Poste = 'Commercial'
16                      AND C.NumDept = E.NumDept);

```

17.

```

1 SELECT NomE
2 FROM EMP
3 WHERE (Poste, Sup) = (SELECT Poste, Sup
4                       FROM EMP
5                       WHERE NomE = 'BERGER')
6    AND NomE != 'BERGER';

```

TP1 : Vues et index

Exercice 1 : vues

1.

```

1 CREATE VIEW V_EMP AS
2 SELECT E.Matr, E.NomE, E.NumDept, (E.Salaire + NVL(E.Commission, 0)) AS
3    GAINS, D.Lieu
4 FROM EMP E
5 JOIN DEPT D ON E.NumDept = D.NumDept;

```

2.

```

1 SELECT * FROM V_EMP
2 WHERE GAINS > 10000;

```

3.

```
1 UPDATE V_EMP
2 SET NomE = 'NOUVEAU_NOM'
3 WHERE NomE = 'MARTIN';
```

4.

```
1 -- Creation de la vue
2 CREATE VIEW VEMP10 AS
3 SELECT * FROM EMP
4 WHERE NumDept = 10;
5
6 -- Insertion
7 INSERT INTO VEMP10 (Matr, NomE, NumDept)
8 VALUES (9999, 'SOULIER', 20);
9
10 -- Recherche via la vue (Ne retournera rien car la vue filtre sur
    NumDept = 10)
11 SELECT * FROM VEMP10
12 WHERE NomE = 'SOULIER';
13
14 -- Recherche via la table (Retournera la ligne inseree)
15 SELECT * FROM EMP
16 WHERE NomE = 'SOULIER';
```

5.

```
1 DROP VIEW VEMP10;
2
3 CREATE VIEW VEMP10 AS
4 SELECT * FROM EMP
5 WHERE NumDept = 10
6 WITH CHECK OPTION;
```

6.

```
1 -- Tentative d'insertion (Echouera a cause de l'option CHECK sur
    NumDept = 10)
2 INSERT INTO VEMP10 (Matr, NomE, NumDept)
3 VALUES (8888, 'BALARD', 30);
4
5 -- Tentative de modification du departement (Echouera egalement)
6 UPDATE VEMP10
7 SET NumDept = 20
8 WHERE Matr = 1234; -- En supposant un matricule existant
```

7.

```
1 -- Solution 1 : En utilisant une vue
2 CREATE VIEW V_TOTAL_SAL_DEPT AS
3 SELECT NumDept, SUM(Salaire) AS TotalSalaire
4 FROM EMP
5 GROUP BY NumDept;
```

```

7 SELECT E.NomE, E.Salaire, (E.Salaire / V.TotalSalaire) * 100 AS
   Pourcentage
8 FROM EMP E
9 JOIN V_TOTAL_SAL_DEPT V ON E.NumDept = V.NumDept;
10
11 -- Solution 2 : En utilisant un select emboite dans le FROM
12 SELECT E.NomE, E.Salaire, (E.Salaire / V.TotalSalaire) * 100 AS
   Pourcentage
13 FROM EMP E
14 JOIN (SELECT NumDept, SUM(Salaire) AS TotalSalaire
15       FROM EMP
16       GROUP BY NumDept) V ON E.NumDept = V.NumDept;

```

Exercice 2 : index

1.

```

1 -- Index avec unicite sur la colonne nome
2 CREATE UNIQUE INDEX idx_emp_nomme ON EMP(NomE);
3
4 -- Index sur la colonne dept (NumDept selon le schema)
5 CREATE INDEX idx_emp_dept ON EMP(NumDept);

```

2.

```

1 DROP INDEX idx_emp_nomme;

```

TP2 : Les Vues en Bases de Données

Exercice 1 : Création d'une Vue Simple

1. & 2. Création et affichage de la vue emp_paris

```

1 CREATE VIEW emp_paris AS
2 SELECT e.emp_id, e.nom, e.salaire
3 FROM Employe e
4 JOIN Departement d ON e.dept_id = d.dept_id
5 WHERE d.ville = 'Paris';
6
7 SELECT * FROM emp_paris;

```

Exercice 2 : Vue Complexe avec Agrégations

Création de la vue stat_dept

```

1 CREATE VIEW stat_dept AS
2 SELECT d.nom_dept,
3        MIN(e.salaire) AS salaire_min,
4        MAX(e.salaire) AS salaire_max,
5        AVG(e.salaire) AS salaire_moyen
6 FROM Employe e
7 JOIN Departement d ON e.dept_id = d.dept_id
8 GROUP BY d.nom_dept;

```

Exercice 3 : WITH CHECK OPTION

Création de la vue emp_lyon

```

1 CREATE VIEW emp_lyon AS
2 SELECT e.*
3 FROM Employe e
4 JOIN Departement d ON e.dept_id = d.dept_id
5 WHERE d.ville = 'Lyon'
6 WITH CHECK OPTION;

```

Exercice 4 : WITH READ ONLY

Création de la vue emp_readonly

```

1 CREATE VIEW emp_readonly AS
2 SELECT * FROM Employe
3 WITH READ ONLY;

```

Exercice 5 : Sécurité et Abstraction

1. Création de la vue emp_public sans le salaire [cite : 1515]

```

1 CREATE VIEW emp_public AS
2 SELECT emp_id, nom, dept_id
3 FROM Employe;

```

2. Discussion sur les performances [cite : 1516]

Non, une vue classique n'améliore pas les performances car elle ne stocke pas les données physiquement. La requête sous-jacente est recalculée et exécutée à chaque appel de la vue.

Exercice 6 : Analyse Globale et Réflexion Approfondie

1. **Architecture basée sur les vues** : Pour limiter l'accès selon le profil, on peut créer des vues spécifiques. Par exemple, une vue utilisant `SYS_CONTEXT` pour filtrer les employés du département du manager connecté, et une vue globale en lecture seule pour le service RH[cite : 1523, 1524].
2. **Limites des vues classiques** : Sur de très grands volumes de données, l'exécution systématique des jointures et calculs sous-jacents de la vue peut dégrader considérablement les temps de réponse.

3. **Intérêt des vues matérialisées** : Elles stockent physiquement les résultats des requêtes, offrant des temps de réponse instantanés pour les consultations fréquentes (idéal pour les RH ou les statistiques globales), bien qu'elles nécessitent une stratégie de rafraîchissement.
4. **Vues vs Privilèges (GRANT/REVOKE)** : Les privilèges contrôlent l'accès global aux objets (tables, vues), tandis que les vues permettent un filtrage granulaire des données (lignes ou colonnes) à l'intérieur de ces objets. Les deux mécanismes s'utilisent conjointement.
5. **Sécurité complète avec les vues ?** Non, les vues ne suffisent pas à elles seules. Si un utilisateur obtient un accès direct aux tables de base, la vue est contournée. Une sécurité robuste nécessite une gestion stricte des privilèges d'accès.
6. **Stratégie d'optimisation** : Combiner des index pertinents sur les clés de jointure et de recherche (ex : dept_id, ville), utiliser le partitionnement pour les très grandes tables, et déployer des vues matérialisées pour les calculs d'agrégation coûteux.

TP : PL/SQL

Exercice Records (Enregistrements)

```

1 DECLARE
2     TYPE table_emp IS TABLE OF employees%ROWTYPE INDEX BY BINARY_INTEGER
3     ;
4     v_emps table_emp;
5 BEGIN
6     FOR i IN 1..3 LOOP
7         v_emps(i).id := i;
8         v_emps(i).nom := 'Employe ' || i;
9         v_emps(i).salaire := 3000 + i * 1000;
10    END LOOP;
11
12    FOR i IN 1..3 LOOP
13        DBMS_OUTPUT.PUT_LINE(v_emps(i).nom || ' gagne ' || v_emps(i).
14                               salaire);
15    END LOOP;
16 END;
17 /

```

Exercice Final de Synthèse

```

1 DECLARE
2     TYPE t_emp_rec IS RECORD (
3         id NUMBER,
4         nom VARCHAR2(50),
5         salaire NUMBER(8,2)
6     );

```

```

7      TYPE t_emp_tab IS TABLE OF t_emp_rec INDEX BY BINARY_INTEGER;
8      v_tab_emps t_emp_tab;
9      v_somme NUMBER := 0;
10     v_max_sal NUMBER := 0;
11 BEGIN
12     v_tab_emps(1).id := 101; v_tab_emps(1).nom := 'Mohamed'; v_tab_emps
        (1).salaire := 4000;
13     v_tab_emps(2).id := 102; v_tab_emps(2).nom := 'Karim'; v_tab_emps
        (2).salaire := 5500;
14     v_tab_emps(3).id := 103; v_tab_emps(3).nom := 'Sara'; v_tab_emps
        (3).salaire := 6000;
15     v_tab_emps(4).id := 104; v_tab_emps(4).nom := 'Salma'; v_tab_emps
        (4).salaire := 4800;
16     v_tab_emps(5).id := 105; v_tab_emps(5).nom := 'Yanis'; v_tab_emps
        (5).salaire := 7200;
17
18     FOR i IN 1..5 LOOP
19         v_somme := v_somme + v_tab_emps(i).salaire;
20         IF v_tab_emps(i).salaire > v_max_sal THEN
21             v_max_sal := v_tab_emps(i).salaire;
22         END IF;
23     END LOOP;
24
25     DBMS_OUTPUT.PUT_LINE('Salaire moyen : ' || (v_somme / 5));
26     DBMS_OUTPUT.PUT_LINE('Salaire maximum : ' || v_max_sal);
27 END;
28 /

```

TP : PL/SQL - Interaction avec les Bases de Données et Structures de Contrôle]

Exercice 1

```

1 CREATE TABLE Nombre (
2     n NUMBER
3 );
4
5 DECLARE
6     v_i NUMBER := 1;
7 BEGIN
8     WHILE v_i <= 100 LOOP
9         INSERT INTO Nombre (n) VALUES (v_i);
10        v_i := v_i + 1;
11    END LOOP;
12 END;
13 /

```

Exercice 2

```
1 DECLARE
2     v_somme NUMBER := 0;
3 BEGIN
4     FOR i IN 1..1000 LOOP
5         v_somme := v_somme + i;
6     END LOOP;
7     DBMS_OUTPUT.PUT_LINE('La somme est : ' || v_somme);
8 END;
9 /
```

Exercice 3

```
1 BEGIN
2     FOR i IN 1..50 LOOP
3         IF MOD(i, 2) = 0 THEN
4             DBMS_OUTPUT.PUT_LINE(i);
5         END IF;
6     END LOOP;
7 END;
8 /
```

Exercice 4

```
1 DECLARE
2     v_n NUMBER := 5; -- Exemple avec n = 5
3     v_fact NUMBER := 1;
4 BEGIN
5     FOR i IN 1..v_n LOOP
6         v_fact := v_fact * i;
7     END LOOP;
8     DBMS_OUTPUT.PUT_LINE('Le factoriel est : ' || v_fact);
9 END;
10 /
```

Exercice 5

```
1 CREATE TABLE Etudiant (
2     id NUMBER,
3     nom VARCHAR2(50),
4     note NUMBER
5 );
6
7 BEGIN
8     FOR i IN 1..10 LOOP
9         INSERT INTO Etudiant (id, nom, note)
10            VALUES (i, 'Etudiant ' || i, 10 + (i * 0.5));
```

```

11     END LOOP;
12 END;
13 /

```

Exercice 6

```

1 DECLARE
2     v_i NUMBER := 10;
3 BEGIN
4     WHILE v_i >= 1 LOOP
5         DBMS_OUTPUT.PUT_LINE(v_i);
6         v_i := v_i - 1;
7     END LOOP;
8 END;
9 /

```

Exercice 7

```

1 CREATE TABLE Produit (
2     id NUMBER,
3     nom VARCHAR2(50),
4     prix NUMBER
5 );
6
7 BEGIN
8     UPDATE Produit
9     SET prix = prix * 1.10;
10 END;
11 /

```

Exercice 8

```

1 BEGIN
2     DELETE FROM Produit
3     WHERE prix < 50;
4 END;
5 /

```

Exercice 9

```

1 DECLARE
2     v_num NUMBER := -15; -- Exemple de nombre
3 BEGIN
4     IF v_num > 0 THEN
5         DBMS_OUTPUT.PUT_LINE('Positif');
6     ELSIF v_num < 0 THEN

```

```

7      DBMS_OUTPUT.PUT_LINE('Negatif');
8  ELSE
9      DBMS_OUTPUT.PUT_LINE('Nul');
10     END IF;
11 END;
12 /

```

Exercice 10

```

1 CREATE TABLE Carre (
2     nombre NUMBER,
3     carre NUMBER
4 );
5
6 BEGIN
7     FOR i IN 1..20 LOOP
8         INSERT INTO Carre (nombre, carre)
9             VALUES (i, i * i);
10    END LOOP;
11 END;
12 /

```

TP : PL-SQL Curseur

Exercice 1

1.

```

1 DECLARE
2     CURSOR cur_liste_commandes IS
3         SELECT Numcmd, Datcmd, Numfour, Mncmd FROM Commande;
4     v_commande cur_liste_commandes%ROWTYPE;
5 BEGIN
6     OPEN cur_liste_commandes;
7     LOOP
8         FETCH cur_liste_commandes INTO v_commande;
9         EXIT WHEN cur_liste_commandes%NOTFOUND;
10        DBMS_OUTPUT.PUT_LINE('Commande : ' || v_commande.Numcmd || ' |
11            Montant : ' || v_commande.Mncmd);
12    END LOOP;
13    CLOSE cur_liste_commandes;
14 END;
15 /

```

2.

```

1 DECLARE
2     CURSOR cur_commandes_importantes IS

```

```

3      SELECT Numcmd, Mncmd FROM Commande
4      WHERE Mncmd > (SELECT AVG(Mncmd) FROM Commande);
5      v_enregistrement cur_commandes_importantes%ROWTYPE;
6  BEGIN
7      OPEN cur_commandes_importantes;
8      LOOP
9          FETCH cur_commandes_importantes INTO v_enregistrement;
10         EXIT WHEN cur_commandes_importantes%NOTFOUND;
11         DBMS_OUTPUT.PUT_LINE('Reference : ' || v_enregistrement.Numcmd
12                               || ' | Valeur : ' || v_enregistrement.Mncmd);
13     END LOOP;
14     CLOSE cur_commandes_importantes;
15 END;
/

```

3.

```

1  DECLARE
2      CURSOR cur_commandes_paris IS
3          SELECT c.Numcmd, c.Mncmd FROM Commande c
4          JOIN Fournisseurs f ON c.Numfour = f.Numfour
5          WHERE f.Adrfour = 'Paris';
6      v_resultat cur_commandes_paris%ROWTYPE;
7  BEGIN
8      OPEN cur_commandes_paris;
9      LOOP
10         FETCH cur_commandes_paris INTO v_resultat;
11         EXIT WHEN cur_commandes_paris%NOTFOUND;
12         DBMS_OUTPUT.PUT_LINE('Numero de commande : ' || v_resultat.
13                               Numcmd);
14     END LOOP;
15     CLOSE cur_commandes_paris;
16 END;
/

```

Exercice 2

1.

```

1  BEGIN
2      FOR ligne_emp IN (SELECT NomE, Comm FROM Emp WHERE Comm IS NOT NULL
3                        ORDER BY Comm DESC) LOOP
4          DBMS_OUTPUT.PUT_LINE(ligne_emp.NomE || ' -> Commission : ' ||
5                                ligne_emp.Comm);
6      END LOOP;
7  END;
/

```

2.

```

1  BEGIN

```

```

2      FOR ligne_emp IN (SELECT NomE FROM Emp WHERE Embauche >= TO_DATE('
      01-09-2006', 'DD-MM-YYYY')) LOOP
3          DBMS_OUTPUT.PUT_LINE('Employe recrute : ' || ligne_emp.NomE);
4      END LOOP;
5  END;
6  /

```

3.

```

1  BEGIN
2      FOR ligne_emp IN (SELECT e.NomE FROM Emp e JOIN Dept d ON e.NumD =
      d.NumD WHERE d.Lieu = 'Cr teil') LOOP
3          DBMS_OUTPUT.PUT_LINE('Employe du site de Creteil : ' ||
      ligne_emp.NomE);
4      END LOOP;
5  END;
6  /

```

4.

```

1  BEGIN
2      FOR ligne_emp IN (SELECT NomE FROM Emp WHERE NumS = (SELECT NumE
      FROM Emp WHERE NomE = 'Guimezanes')) LOOP
3          DBMS_OUTPUT.PUT_LINE('Subordonne : ' || ligne_emp.NomE);
4      END LOOP;
5  END;
6  /

```

5.

```

1  DECLARE
2      v_total_salaires NUMBER := 0;
3      v_effectif NUMBER := 0;
4  BEGIN
5      FOR ligne_emp IN (SELECT Salaire FROM Emp) LOOP
6          v_total_salaires := v_total_salaires + NVL(ligne_emp.Salaire,
7              0);
8          v_effectif := v_effectif + 1;
9      END LOOP;
10     IF v_effectif > 0 THEN
11         DBMS_OUTPUT.PUT_LINE('Salaire moyen : ' || (v_total_salaires /
12             v_effectif));
13     END IF;
14 END;
15 /

```

6.

```

1  DECLARE
2      v_nombre_commissions NUMBER := 0;
3  BEGIN
4      FOR ligne_emp IN (SELECT Comm FROM Emp WHERE Comm IS NOT NULL) LOOP
5          v_nombre_commissions := v_nombre_commissions + 1;
6      END LOOP;

```

```

7      DBMS_OUTPUT.PUT_LINE('Nombre d''employes avec commission : ' ||
      v_nombre_commissions);
8  END;
9  /

```

7.

```

1  BEGIN
2      FOR ligne_emp IN (SELECT NomE, Salaire FROM Emp WHERE Salaire > (
      SELECT AVG(Salaire) FROM Emp)) LOOP
3          DBMS_OUTPUT.PUT_LINE(ligne_emp.NomE || ' <- Salaire : ' ||
      ligne_emp.Salaire);
4      END LOOP;
5  END;
6  /

```

TP : Procédures et fonctions

Exercice 1

1.

```

1  BEGIN
2      DBMS_OUTPUT.PUT_LINE('Hello');
3  END;
4  /

```

2.

```

1  CREATE OR REPLACE FUNCTION calculer_carre (p_valeur IN NUMBER)
2  RETURN NUMBER IS
3  BEGIN
4      RETURN p_valeur * p_valeur;
5  END;
6  /

```

Exercice 2

Procédure avec paramètre OUT

```

1  CREATE OR REPLACE PROCEDURE max_deux_nombres (
2      p_num1 IN NUMBER,
3      p_num2 IN NUMBER,
4      p_max OUT NUMBER
5  ) IS
6  BEGIN
7      IF p_num1 > p_num2 THEN
8          p_max := p_num1;
9      ELSE
10         p_max := p_num2;

```

```

11     END IF;
12 END;
13 /

```

Exercice 3

Procédure avec paramètres IN OUT

```

1 CREATE OR REPLACE PROCEDURE permuter_nombres (
2     p_a IN OUT NUMBER,
3     p_b IN OUT NUMBER
4 ) IS
5     v_temp NUMBER;
6 BEGIN
7     v_temp := p_a;
8     p_a := p_b;
9     p_b := v_temp;
10 END;
11 /

```

Exercice 4

Procédure basée sur le nom (avec ROWNUM = 1 pour éviter les doublons)

```

1 CREATE OR REPLACE PROCEDURE augmenter_salaire (
2     p_nom IN VARCHAR2,
3     p_taux IN NUMBER
4 ) IS
5 BEGIN
6     UPDATE EMPLOYES
7     SET SALARY = SALARY + (SALARY * p_taux / 100)
8     WHERE ENAME = p_nom
9     AND ROWNUM = 1;
10 END;
11 /

```

Variante avec la clé primaire (ENUM)

```

1 CREATE OR REPLACE PROCEDURE augmenter_salaire_enum (
2     p_enum IN VARCHAR2,
3     p_taux IN NUMBER
4 ) IS
5 BEGIN
6     UPDATE EMPLOYES
7     SET SALARY = SALARY + (SALARY * p_taux / 100)
8     WHERE ENUM = p_enum;
9 END;
10 /

```

Exercice 5

Fonction de conversion avec arrondi

```
1 CREATE OR REPLACE FUNCTION convertir_mad_en_eur (p_montant_mad IN
  NUMBER)
2 RETURN NUMBER IS
3     v_taux CONSTANT NUMBER := 11;
4 BEGIN
5     RETURN ROUND(p_montant_mad / v_taux, 2);
6 END;
7 /
```

Exercice 6

Procédure avec curseur implicite FOR

```
1 CREATE OR REPLACE PROCEDURE afficher_salaires_sup (p_montant IN NUMBER)
  IS
2 BEGIN
3     FOR r_emp IN (SELECT ENAME, SALARY FROM EMPLOYEES WHERE SALARY >
4                   p_montant) LOOP
5         DBMS_OUTPUT.PUT_LINE(r_emp.ENAME || ' gagne ' || r_emp.SALARY);
6     END LOOP;
7 END;
8 /
```

Exercice 7

Fonction avec gestion d'exception

```
1 CREATE OR REPLACE FUNCTION salaire_moyen_dept (p_dept IN VARCHAR2)
2 RETURN NUMBER IS
3     v_moyenne NUMBER;
4 BEGIN
5     SELECT AVG(SALARY) INTO v_moyenne
6     FROM EMPLOYEES
7     WHERE DEPT = p_dept;
8
9     RETURN NVL(v_moyenne, 0);
10 EXCEPTION
11     WHEN NO_DATA_FOUND THEN
12         RETURN NULL;
13 END;
14 /
```

Exercice 8

Procédure avec vérification de non-nullité

```

1 CREATE OR REPLACE PROCEDURE maj_adresse_nom (
2     p_nom IN VARCHAR2,
3     p_nouv_adresse IN VARCHAR2
4 ) IS
5 BEGIN
6     IF p_nouv_adresse IS NOT NULL AND TRIM(p_nouv_adresse) != '' THEN
7         UPDATE EMPLOYES
8         SET DRESS = p_nouv_adresse
9         WHERE ENAME = p_nom
10            AND ROWNUM = 1;
11     END IF;
12 END;
13 /

```

Exercice 9

1. Fonction retournant l'effectif

```

1 CREATE OR REPLACE FUNCTION nb_employees_dept (p_dept IN VARCHAR2)
2 RETURN NUMBER IS
3     v_count NUMBER;
4 BEGIN
5     SELECT COUNT(*) INTO v_count
6     FROM EMPLOYES
7     WHERE DEPT = p_dept;
8
9     RETURN v_count;
10 END;
11 /

```

2. Procédure exploitant la fonction pour chaque département

```

1 CREATE OR REPLACE PROCEDURE afficher_nb_emp_par_dept IS
2 BEGIN
3     FOR r_dept IN (SELECT DISTINCT DEPT FROM EMPLOYES WHERE DEPT IS NOT
4                     NULL) LOOP
5         DBMS_OUTPUT.PUT_LINE('Departement : ' || r_dept.DEPT ||
6                               ' | Effectif : ' || nb_employees_dept(
7                                   r_dept.DEPT));
8     END LOOP;
9 END;
10 /

```

Exercice 10

Procédure de suppression

```

1 CREATE OR REPLACE PROCEDURE supprimer_petits_salaires (p_montant IN
2     NUMBER) IS
3 BEGIN
4     DELETE FROM EMPLOYES

```

```

4      WHERE SALARY < p_montant;
5
6      DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' employe(s) supprime(s).');
7  END;
8  /

```

TP2 - Curseurs

1

```

1  DECLARE
2      v_petit_nbr NUMBER := 25;
3      v_grand_nbr NUMBER := 5;
4      v_intermed NUMBER;
5  BEGIN
6      IF v_petit_nbr > v_grand_nbr THEN
7          v_intermed := v_petit_nbr;
8          v_petit_nbr := v_grand_nbr;
9          v_grand_nbr := v_intermed;
10     END IF;
11     DBMS_OUTPUT.PUT_LINE('Valeur minimale = ' || v_petit_nbr);
12     DBMS_OUTPUT.PUT_LINE('Valeur maximale = ' || v_grand_nbr);
13 END;
14 /

```

2

```

1  DECLARE
2      v_nombre NUMBER := 3;
3  BEGIN
4      IF MOD(v_nombre, 2) = 0 THEN
5          DBMS_OUTPUT.PUT_LINE('Le nombre ' || v_nombre || ' est pair. ');
6      ELSE
7          DBMS_OUTPUT.PUT_LINE('Le nombre ' || v_nombre || ' est impair. ');
8      END IF;
9  END;
10 /

```

3

```

1  DECLARE
2      v_date_reference DATE := TO_DATE('02-Mar-2024', 'DD-Mon-YYYY');
3      v_nom_jour VARCHAR2(20);
4  BEGIN

```

```

5      v_nom_jour := TO_CHAR(v_date_reference, 'DAY', 'NLS_DATE_LANGUAGE=
      ENGLISH');
6
7      IF TRIM(v_nom_jour) IN ('SATURDAY', 'SUNDAY') THEN
8          DBMS_OUTPUT.PUT_LINE('La date correspond au jour ' || TRIM(
          v_nom_jour) || ' : week-end.');
```

```

9      ELSE
10         DBMS_OUTPUT.PUT_LINE('La date correspond au jour ' || TRIM(
          v_nom_jour) || ' : jour ouvrable.');
```

```

11     END IF;
12 END;
13 /
```

4

```

1 BEGIN
2     FOR ligne_client IN (
3         SELECT client_id, nom, ville, age
4         FROM clients
5     ) LOOP
6         INSERT INTO tmp_clients (client_id, nom, ville, age)
7         VALUES (
8             ligne_client.client_id,
9             ligne_client.nom,
10            ligne_client.ville,
11            ligne_client.age
12        );
13    END LOOP;
14
15    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' enregistrement(s) transfere(
      s).');
```

```

16 END;
17 /
```

Curseur 1

```

1 DECLARE
2     CURSOR cur_liste_pays IS
3         SELECT nom FROM pays;
4 BEGIN
5     FOR ligne_pays IN cur_liste_pays LOOP
6         DBMS_OUTPUT.PUT_LINE(ligne_pays.nom);
7     END LOOP;
8 END;
9 /
```

Curseur 2

```

1 DECLARE
2     CURSOR cur_infos_pays IS
3         SELECT pays_id, nom FROM pays;
4 BEGIN
5     DBMS_OUTPUT.PUT_LINE('Code Pays | Designation');
6     DBMS_OUTPUT.PUT_LINE('-----+-----');
7
8     FOR ligne_pays IN cur_infos_pays LOOP
9         DBMS_OUTPUT.PUT_LINE(ligne_pays.pays_id || ' | ' || ligne_pays.
10                                nom);
11     END LOOP;
12 END;
13 /

```

Curseur 3

```

1 BEGIN
2     UPDATE pays
3     SET nom = UPPER(nom);
4
5     DBMS_OUTPUT.PUT_LINE('Nombre de lignes modifiees : ' || SQL%
6                             ROWCOUNT);
7 END;
8 /

```

Curseur 4

```

1 DECLARE
2     CURSOR cur_employes_departements IS
3         SELECT e.id, e.nom, e.prenom, d.nom AS intitule_departement
4         FROM employees e
5         JOIN departements d ON e.departement_id = d.dep_id;
6 BEGIN
7     FOR ligne_emp IN cur_employes_departements LOOP
8         DBMS_OUTPUT.PUT_LINE(
9             ligne_emp.id || ' | ' ||
10             ligne_emp.nom || ' ' ||
11             ligne_emp.prenom || ' | ' ||
12             ligne_emp.intitule_departement
13         );
14     END LOOP;
15 END;
16 /

```

Curseur 5

```

1 BEGIN
2     DELETE FROM pays
3     WHERE pays_id = 1001;
4
5     IF SQL%FOUND THEN
6         DBMS_OUTPUT.PUT_LINE('Le pays a ete supprime avec succes.');
```

```

7     ELSE
8         DBMS_OUTPUT.PUT_LINE('Aucun pays correspondant trouve.');
```

```

9     END IF;
10 END;
11 /
```

Curseur 6

```

1 BEGIN
2     UPDATE pays
3     SET nom = 'Non renseigne'
4     WHERE pays_id = 9999;
5
6     IF SQL%NOTFOUND THEN
7         DBMS_OUTPUT.PUT_LINE('Aucune modification effectuee.');
```

```

8     ELSE
9         DBMS_OUTPUT.PUT_LINE('Mise a jour realisee avec succes.');
```

```

10    END IF;
11 END;
12 /
```

Curseur 7

```

1 DECLARE
2     CURSOR cur_revision_salaires IS
3         SELECT id, salaire
4         FROM employees
5         WHERE departement_id = 8
6         FOR UPDATE;
7 BEGIN
8     FOR ligne_emp IN cur_revision_salaires LOOP
9         IF ligne_emp.salaire < 5000 THEN
10             UPDATE employees
11             SET salaire = salaire + 100
12             WHERE CURRENT OF cur_revision_salaires;
13         ELSE
14             UPDATE employees
15             SET salaire = salaire + 50
16             WHERE CURRENT OF cur_revision_salaires;
17         END IF;
18     END LOOP;
19
```

```

20 DBMS_OUTPUT.PUT_LINE('Revalorisation des salaires terminee.');
```

```

21 END;
```

```

22 /
```

TP - Triggers

Création des Tables

```

1 DROP TABLE passer CASCADE CONSTRAINTS;
```

```

2 DROP TABLE examen CASCADE CONSTRAINTS;
```

```

3 DROP TABLE inscription CASCADE CONSTRAINTS;
```

```

4 DROP TABLE prerequis CASCADE CONSTRAINTS;
```

```

5 DROP TABLE module CASCADE CONSTRAINTS;
```

```

6 DROP TABLE etudiant CASCADE CONSTRAINTS;
```

```

7
```

```

8 CREATE TABLE etudiant (
```

```

9     id_etudiant NUMBER PRIMARY KEY,
```

```

10    nom VARCHAR2(50),
```

```

11    prenom VARCHAR2(50),
```

```

12    date_inscription DATE,
```

```

13    moyenne NUMBER
```

```

14 );
```

```

15
```

```

16 CREATE TABLE module (
```

```

17     code_module VARCHAR2(10) PRIMARY KEY,
```

```

18     intitule VARCHAR2(100),
```

```

19     effecMax NUMBER,
```

```

20     note_min NUMBER
```

```

21 );
```

```

22
```

```

23 CREATE TABLE prerequis (
```

```

24     module VARCHAR2(10) REFERENCES module(code_module),
```

```

25     prerequis VARCHAR2(10) REFERENCES module(code_module),
```

```

26     note_min NUMBER,
```

```

27     PRIMARY KEY (module, prerequis)
```

```

28 );
```

```

29
```

```

30 CREATE TABLE inscription (
```

```

31     id_etudiant NUMBER REFERENCES etudiant(id_etudiant),
```

```

32     code_module VARCHAR2(10) REFERENCES module(code_module),
```

```

33     date_inscription DATE,
```

```

34     PRIMARY KEY (id_etudiant, code_module)
```

```

35 );
```

```

36
```

```

37 CREATE TABLE examen (
```

```

38     id_examen NUMBER PRIMARY KEY,
```

```

39     code_module VARCHAR2(10) REFERENCES module(code_module),
```

```

40     date_examen DATE,
```

```

41     coefficient NUMBER
42 );
43
44 CREATE TABLE passer (
45     id_etudiant NUMBER REFERENCES etudiant(id_etudiant),
46     id_examen NUMBER REFERENCES examen(id_examen),
47     note NUMBER,
48     PRIMARY KEY (id_etudiant, id_examen)
49 );

```

1 - Interdiction de modification de la note requise

```

1 CREATE OR REPLACE TRIGGER trg_interdire_modif_note_requise
2 BEFORE UPDATE OF note_min ON prerequis
3 FOR EACH ROW
4 BEGIN
5     RAISE_APPLICATION_ERROR(
6         -20001,
7         'La note minimale requise ne peut pas etre modifiee.'
8     );
9 END;
10 /

```

2 - Vérification de la capacité du module

```

1 CREATE OR REPLACE TRIGGER trg_verifier_capacite_module
2 BEFORE INSERT ON inscription
3 FOR EACH ROW
4 DECLARE
5     v_nombre_inscriptions NUMBER;
6     v_capacite_autorisee NUMBER;
7 BEGIN
8     SELECT effecMax
9     INTO v_capacite_autorisee
10    FROM module
11   WHERE code_module = :NEW.code_module;
12
13     SELECT COUNT(*)
14     INTO v_nombre_inscriptions
15    FROM inscription
16   WHERE code_module = :NEW.code_module;
17
18     IF v_nombre_inscriptions >= v_capacite_autorisee THEN
19         RAISE_APPLICATION_ERROR(
20             -20002,
21             'Ajout refuse : capacite maximale atteinte pour le module '
22             || :NEW.code_module
23         );
24     END IF;

```

```
24 END;  
25 /
```

3 - Contrôle de création d'examen

```
1 CREATE OR REPLACE TRIGGER trg_controle_creation_examen  
2 BEFORE INSERT ON examen  
3 FOR EACH ROW  
4 DECLARE  
5     v_total_inscrits NUMBER;  
6 BEGIN  
7     SELECT COUNT(*)  
8     INTO v_total_inscrits  
9     FROM inscription  
10    WHERE code_module = :NEW.code_module;  
11  
12    IF v_total_inscrits = 0 THEN  
13        RAISE_APPLICATION_ERROR(  
14            -20003,  
15            'Creation impossible : aucun etudiant inscrit dans le  
16              module ' || :NEW.code_module  
17        );  
18    END IF;  
19 END;  
20 /
```

4 - Vérification de la date d'inscription

```
1 CREATE OR REPLACE TRIGGER trg_verifier_date_inscription  
2 BEFORE INSERT ON passer  
3 FOR EACH ROW  
4 DECLARE  
5     v_date_enregistrement DATE;  
6     v_date_epreuve DATE;  
7 BEGIN  
8     SELECT date_inscription  
9     INTO v_date_enregistrement  
10    FROM inscription  
11    WHERE id_etudiant = :NEW.id_etudiant  
12    AND code_module = (  
13        SELECT code_module  
14        FROM examen  
15        WHERE id_examen = :NEW.id_examen  
16    );  
17  
18    SELECT date_examen  
19    INTO v_date_epreuve  
20    FROM examen  
21    WHERE id_examen = :NEW.id_examen;
```

```

22
23     IF v_date_enregistrement >= v_date_epreuve THEN
24         RAISE_APPLICATION_ERROR(
25             -20004,
26             'Inscription invalide : la date d''inscription est
                posterieure ou egale a la date de l''examen.'
27         );
28     END IF;
29 END;
30 /

```

5 - Détection de cycle dans les prérequis

```

1  CREATE OR REPLACE FUNCTION verifier_boucle_prerequis(
2      p_module_source VARCHAR2,
3      p_module_cible VARCHAR2
4  )
5  RETURN BOOLEAN
6  IS
7      CURSOR cur_dependances IS
8          SELECT prerequis
9          FROM prerequis
10         WHERE module = p_module_source;
11 BEGIN
12     FOR ligne_prerequis IN cur_dependances LOOP
13         IF ligne_prerequis.prerequis = p_module_cible THEN
14             RETURN TRUE;
15         END IF;
16
17         IF verifier_boucle_prerequis(ligne_prerequis.prerequis,
18             p_module_cible) THEN
19             RETURN TRUE;
20         END IF;
21     END LOOP;
22
23     RETURN FALSE;
24 END;
25 /
26
27 CREATE OR REPLACE TRIGGER trg_detection_cycle
28 BEFORE INSERT OR UPDATE ON prerequis
29 FOR EACH ROW
30 BEGIN
31     IF verifier_boucle_prerequis(:NEW.prerequis, :NEW.module) THEN
32         RAISE_APPLICATION_ERROR(
33             -20005,
34             'Cycle detecte entre les modules : ' || :NEW.module || ' et
                ' || :NEW.prerequis
35         );
36     END IF;

```

```

36 END;
37 /

```

6 - Vérification des prérequis de l'étudiant

```

1  CREATE OR REPLACE TRIGGER trg_verifier_prerequis_etudiant
2  BEFORE INSERT ON inscription
3  FOR EACH ROW
4  DECLARE
5      CURSOR cur_conditions IS
6          SELECT p.prerequis, p.note_min
7          FROM prerequis p
8          WHERE p.module = :NEW.code_module;
9      v_meilleure_note NUMBER;
10 BEGIN
11     FOR ligne_condition IN cur_conditions LOOP
12         BEGIN
13             SELECT MAX(pa.note)
14             INTO v_meilleure_note
15             FROM passer pa
16             JOIN examen e ON pa.id_examen = e.id_examen
17             WHERE pa.id_etudiant = :NEW.id_etudiant
18             AND e.code_module = ligne_condition.prerequis;
19         EXCEPTION
20             WHEN NO_DATA_FOUND THEN
21                 v_meilleure_note := NULL;
22         END;
23
24         IF v_meilleure_note IS NULL OR v_meilleure_note <
25            ligne_condition.note_min THEN
26             RAISE_APPLICATION_ERROR(
27                 -20006,
28                 'Condition non satisfaite pour le prerequis ' ||
29                 ligne_condition.prerequis || '. Note minimale
30                 attendue : ' || ligne_condition.note_min
31             );
32         END IF;
33     END LOOP;
34 END;
35 /

```

7 - Recalcul de la moyenne

```

1  CREATE OR REPLACE TRIGGER trg_recalculer_moyenne
2  AFTER INSERT OR UPDATE OR DELETE ON passer
3  FOR EACH ROW
4  DECLARE
5      v_etudiant_concerne NUMBER;
6      v_moyenne_calculee NUMBER;

```

```

7      v_nombre_notes NUMBER;
8  BEGIN
9      IF INSERTING OR UPDATING THEN
10         v_etudiant_concerne := :NEW.id_etudiant;
11     ELSE
12         v_etudiant_concerne := :OLD.id_etudiant;
13     END IF;
14
15     SELECT SUM(pa.note * e.coefficient) / SUM(e.coefficient), COUNT(*)
16     INTO v_moyenne_calculee, v_nombre_notes
17     FROM passer pa
18     JOIN examen e ON pa.id_examen = e.id_examen
19     WHERE pa.id_etudiant = v_etudiant_concerne
20     AND pa.note IS NOT NULL;
21
22     IF v_nombre_notes > 0 THEN
23         UPDATE etudiant
24         SET moyenne = v_moyenne_calculee
25         WHERE id_etudiant = v_etudiant_concerne;
26     ELSE
27         UPDATE etudiant
28         SET moyenne = NULL
29         WHERE id_etudiant = v_etudiant_concerne;
30     END IF;
31 END;
32 /

```

8 - Contrôle de la nouvelle note minimale

```

1  CREATE OR REPLACE TRIGGER trg_controle_nouvelle_note_min
2  BEFORE UPDATE OF note_min ON prerequis
3  FOR EACH ROW
4  DECLARE
5      CURSOR cur_etudiants_inscrits IS
6          SELECT DISTINCT i.id_etudiant
7          FROM inscription i
8          WHERE i.code_module = :NEW.module;
9      v_note_reference NUMBER;
10 BEGIN
11     FOR ligne_etudiant IN cur_etudiants_inscrits LOOP
12         SELECT MAX(pa.note)
13         INTO v_note_reference
14         FROM passer pa
15         JOIN examen e ON pa.id_examen = e.id_examen
16         WHERE pa.id_etudiant = ligne_etudiant.id_etudiant
17         AND e.code_module = :NEW.prerequis;
18
19         IF v_note_reference IS NULL OR v_note_reference < :NEW.note_min
20             THEN
21             RAISE_APPLICATION_ERROR(

```

```

21         -20008,
22         'La nouvelle note minimale rend invalide le dossier de
           l''etudiant ' || ligne_etudiant.id_etudiant
23     );
24     END IF;
25     END LOOP;
26 END;
27 /

```

9 - Contrôle de l'effectif du module

```

1  CREATE OR REPLACE TRIGGER trg_controle_effectif_module
2  BEFORE UPDATE OF effecMax ON module
3  FOR EACH ROW
4  DECLARE
5      v_nombre_actuel NUMBER;
6  BEGIN
7      SELECT COUNT(*)
8      INTO v_nombre_actuel
9      FROM inscription
10     WHERE code_module = :NEW.code_module;
11
12     IF :NEW.effecMax < v_nombre_actuel THEN
13         RAISE_APPLICATION_ERROR(
14             -20009,
15             'Modification refusee : effectif maximal inferieur au
               nombre actuel d''inscriptions.'
16         );
17     END IF;
18 END;
19 /

```